Section 1: Project Overview

In this project, you will use Rust to build a simple reduction program for terms in the λ calculus (**Section 2**). Most of the code is provided for you -- see the file `lambda.tar.gz` on Canvas. You will only need to implement a few key procedures:

- `parse.rs` -- `or` This is a parser combinator (**Section 3**), that you will have to implement. For an example of a similar implementation, see the definition of `and` in the same file. This is part of the front-end of our λ calculus implementation.
- `term.rs` -- `normal_order_reduction` This reduces a term to weak head normal form using the normal order reduction strategy (**Section 2**). This is part of the back-end of our λ calculus implementation.
- `main.rs` -- `application_parser` This is an "unwrapped" parser that parses function applications. You don't need to test this function.

In addition, you must add proper test cases for the procedures that you implement, except for `application_parser`.

The starting code is in the file `lambda.tar.gz`, which is available in this assignment description on Canvas. Once you have extracted that file's contents, you can compile it on Onyx with `make`, and test it with `make test`. There are chunks of code marked `todo!(...)` -- these are the items that you must complete.

Section 2: λ Calculus

The untyped λ calculus was developed by Alonzo Church in the 1930s with the intention of serving as a formal proof system. However, as a proof system, λ calculus is not sound (i.e., it can simultaneously prove A and $\neg A$). This issue led to the development of simply typed λ calculus, and later System F, which *are* sound...but we digress!

It is impossible to understand contemporary issues in programming languages without understanding λ calculus. The untyped λ calculus that Church introduced influenced McCarthy, at least partially, as he developed Lisp. In the 70s, Dana Scott's work, particularly "Data Types as Lattices", provided the first-ever formal semantics for any programming language. His language was called LAMBDA, which was based on the λ calculus. Meanwhile, Peter Landin's papers (e.g., "The Next 700 Programming Languages", 1965) predicted that λ calculus would have tremendous influence on the design of programming languages for years to come.

A. Syntax

Here's a BNF grammar that describes the variation of λ calculus that we consider:

```
term : <identifier>
       | "(" <term> <term> ")"
       | "\" <identifier> "." <term>
```

Where an `identifier` matches the regular expression `[a-zA-Z][a-zA-Z0-9]*`.
Note: "\" is usually written as λ, but that's hard to type on your keyboard, so we'll use "\" instead.

i. Examples

This is a valid term in our λ calculus:

```
foo
```

Here's the function that simply returns its argument, called the *identity* function:

```
\x.x
```

Here's applying the identity function to itself:

```
(\x.x \x.x)
```

You might be able to guess that applying the identity function to itself reduces to just the identity function. We describe this reduction using the notation `(\x.x \x.x)` $\rightarrow_{\beta}^{*}$ `\x.x` .

Free Variables

In our λ calculus, a variable is *free* if it is not bound as a function parameter. For example, the term `foo` is a free variable. But in the term `\foo.foo`, `foo` is not a free variable because it is a parameter of the λ.

Now, let's discuss how reduction works, i.e., its *semantics*.

B. Semantics

The meaning, or semantics, of a term in λ calculus is defined by how it is reduced. Reductions occur using β-reduction, which involves applying functions to arguments.

i. β-reduction

To perform β-reduction for a term in the form `(\x.M N)`, we simply substitute every free occurrence of `x` in `M` with `N`. For example, `(\x.y z)` β-reduces to `y`, since `x` does not occur in the body of `\x.y`. We write this as `(\x.y z)` $\rightarrow_\beta^*$ `y`.

Here's a more interesting example. Let's say that `TRUE` $\triangleq$ `\x.\y.x`. Note that the syntax `a` $\triangleq$ `b` isn't part of our λ calculus. For this document, we'll just agree that whenever we use this notation, we'll replace `a` with `b` later.

We can define `FALSE` $\triangleq$ `\x.\y.y`. Now, let's define `AND` $\triangleq$ `\x.\y.((x y) FALSE)`. Let's test this definition out:
`((AND TRUE) TRUE)` $=$ `((\x.\y.((x y) FALSE) TRUE) TRUE)` $\rightarrow_\beta$ `(\y.((TRUE y) FALSE) TRUE)` $\rightarrow_\beta$ `((TRUE TRUE) FALSE)` $=$ `((\x.\y.x TRUE) FALSE)` $\rightarrow_\beta$ `(\y.TRUE FALSE)` $\rightarrow_\beta$ `TRUE`.

Try a few other applications of `AND` by yourself.

There's one small complication. Consider `((\x.\y.x y) z)`. If we perform β-reduction naively, we'll reduce this to `(\y.y z)`, and then to `z`. But that doesn't seem right. The term `((\x.\y.x y) z)` should be equivalent (we'll formalize this in a moment) to `((\x0.\y0.x0 y) z)` -- after all, we only renamed function parameters, which shouldn't affect the lambda. But that term reduces to `(\y0.y z)`, and then reduces to `y`. Obviously, we need to practice a little care to avoid inadvertently binding free variables!

ii. α-conversion

This problematic situation can only arise in terms of the form `(f t)` when `t` contains a free variable that is bound in `f`. This problem can be avoided via α-conversion. α-conversion is the process of taking a term and renaming a bound variable. For example, in `\x.x`, `x` can be renamed to `y`, thus yielding `\y.y`. If a term T_1 can be converted into T_2 by applying α-conversion, we say that $T_1 =_\alpha T_2$, i.e., T_1 and T_2 are α - equivalent. So, `\x.x` $=_\alpha$ `\y.y`. We must apply α-conversion before β-reduction for any term that causes this problematic behavior.

iii. Reduction Strategies

A term is in *normal form* if it nor any of its subterms are in the form `(\x.P Q)`. We're interested in finding the normal forms of terms because that's how computation works in λ calculus. There are a couple facts about normal forms that you should know:

- Not all terms have a normal form. This is a problem because trying to β-reduce such terms to a normal form is impossible.
- "Does a term have a normal form?" is undecidable. This is a problem because we can't know if β-reduction will ever complete for a given term.

- But if a term *does* have a normal form, then it is unique. That is, no matter which subterm we apply β-reduction to first, we end up with the same normal form.
  A *reduction strategy* describes which term we choose to β-reduce first.

As an example, consider this term: $\Omega \triangleq$ `(\x.(x x) \x.(x x))`. Let's try to β-reduce Ω:
`(\x.(x x) \x.(x x)) = (\x.(x x) \x.(x x))`
If we keep applying β-reduction, we'll keep getting the same term. Thus, Ω has no normal form.

This phenomenon has numerous practical implications. For example, consider this definition of `IF`, which is intended to denote our usual notion of control flow: `IF` $\triangleq$ `\condition.\then.\else.((condition then) else)`. Here, `condition` must be a term that reduces to `TRUE` or `FALSE`. If `condition` reduces to `TRUE`, then the result of the reduction is `then`, since `TRUE` reduces to its first parameter. On the other hand, if `condition` reduces to `FALSE`, then the result of the reduction is `else` since `FALSE` reduces to its second parameter.

Consider another example: `(((IF TRUE) TRUE) (\x.(x x) \x.(x x)))`. This term has a normal form of `TRUE`. But the reduction strategy we have to use to find that differs from what we're accustomed to in languages like C or Java. In those languages, to evaluate `f(x1, ..., xn)`, we must first evaluate `x1 ... xn` and then call `f` with those values. Thus, the evaluation of `x1 ... xn` occurs prior to control flow transferring to `f`. This is the *applicative order* reduction strategy. Applicative order doesn't work very well for λ calculus because it fails to reduce terms like `(((IF TRUE) TRUE) (\x.(x x) \x.(x x)))`.

It's a good time to say that our exact goal isn't to convert into normal form because λ calculus terms are allowed to include free variables. For example, the term `(A (\x.(x x) \x.(x x)))` is not in normal form because it contains a term of the form `(\x.Q M)`. But `A` *could* be the term `\x.y`, in which case the whole term β-reduces to y. So, we only want our term to be in *weak head normal form*, meaning that the outer-most left-most term **is not** in the form `(\x.P Q)`.

To obtain a term in weak head normal form, we can simply apply β-reduction to the head position of every term until we can't any longer. This reduction strategy is called *normal order reduction*. It will always find the weak head normal form if one exists. It's basically like lazy evaluation -- a term is only reduced if it eventually becomes a topmost function application. Otherwise, it may never need to be reduced -- so we put off reducing it!

iv. Reduction Algorithm

This pseudo code shows the algorithm to reduce a term to weak head normal form:

```
(** Declares a new sum-type called term. **)
type term =
    | Var : string -> term
```

```
    | Lambda : string -> term -> term
    | Application : term -> term -> term

(** Returns the set of free variables appearing in `t`.
    Note the syntax (bound_variables : set string = {}) indicates that
    `bound_variables` is an optional argument of type `set string`.
    If it is not provided, its value is the empty set. **)
let rec free_variables (t : term) (bound_variables : set string = {})
    : set string
    =
    match t with
    | Var(v) -> if v ∉ bound_variables then {v} else {}
    | Application(f, p) ->
        (free_variables f bound_variables) ∪ (free_variables p
bound_variables)
    | Lambda(param, body) ->
        free_variables body (bound_variables ∪ {param})

(** Returns the set of all variables that appear in `t`. **)
let rec all_variables (t : term) : set string =
    match t with
    | Var(v) -> {v}
    | Application(f, p) -> (all_variables f) ∪ (all_variables p)
    | Lambda(param, body) -> {param} ∪ (all_variables body)

(** Returns a string based on `name` that does not appear in `all_names`.
**)
let rec fresh_var (name : string) (all_names : set string) : string =
    <implementation not shown>

(** Substitutes all free occurences of `v` with `x` in `t`.
    Inputs: t, the term to perform substitution on.
            v, the name of the variable to replace.
            x, the term to substitute in for v.
    Outputs: t with `x` substituted for `v`. **)
let rec substitute (t : term) (v : string) (x : term) =
    match t with
    | Var(name) -> if name = v then x else Var(name)
    | Application(f, p) -> Application(
        substitute f v x,
        substitute p v x,
      )
    | Lambda(param, body) ->
        if param = v then
            (* In this case, `v` is not free in body, so we're done. *)
            Lambda(param, body)
```

```
        else if param ∈ free_variables x then
            (* In this case, we need to rename `param`
               (i.e., do α conversion) or else there
               will be incorrect substitutions of `param` in `x`. *)
             let vars = (all_variables t) ∪ (all_variables x) in
             let new_param = fresh_var param vars in
             let new_body = substitute body param (Var new_param) in
             Lambda(new_param, substitute new_body v x)
        else
            Lambda(param, substitute body v x)

(** A function that reduces terms to weak head normal form using
    normal order reduction.
    Inputs: t, a term.
    Outputs: the term in weak head normal form. **)
let rec normal_order_reduction (t : term) : term =
    match t with
    | Var(_) -> t
    | Lambda(_, _) -> t
    | Application(f, p) -> begin
        match normal_order_reduction f with
        | Lambda(param, body) -> normal_order_reduction (substitute body
param p)
        | g -> Application(g, p)
    end
```

Note that this is pseudo code from a functional programming language. It will not work unmodified in Rust!

Section 3: Parser Combinators

Parser combinators are an incredibly simple yet powerful idea. Before discussing them, let's ask a simple question: "What *is* a parser?"

A parser is a program that transforms an input string into a resulting syntax tree and the unparsed remainder of the string. Using the same syntax that we used in our algorithms, we can write that idea as `type parser (t : Type) = string -> option (t, string)`. Here, `t` is a type parameter that stands in for the data type representing our syntax tree. For our λ calculus, a parser has the type `parser term`. For Lisp, a parser has the type `parser sexpr`. The final result of parsing is stored in an `option` type that holds either `Some (x, y)` when parsing succeeds with the syntax tree `x` and a remaining string `y`, or `None` when parsing fails.

A *parser combinator* is a program that combines two parsers. For example, we might want a parser that allows either the language recognized by the parser `p1` or the parser `p2`:

```
type either a b =
    | Left  : a -> either a b
    | Right : b -> either a b

(** A parser combinator recognizing the union of p1 and p2. **)
let parser_or (t1 t2 : Type) (p1 : parser t1) (p2 : parser t2)
    : parser (either t1 t2)
    =
    λ (str : string) ->
    match p1 str with
    | Some (x, y) -> Some (Left x, y)
    | None -> begin
        match p2 str with
        | Some (x, y) -> Some (Right x, y)
        | None -> None
      end
```

In the file `parse.rs`, there are several parser combinators. Make sure you understand how each one works.

Section 4: Rubric

Component	Points
`parse.rs/or`	10
`term.rs/normal_order_reduction`	10
`main.rs/application_parser`	10
Test cases	20
Total	50